Fire, Heat, and the Wildlife Recovery Gap in Australia

Does post-fire heat exposure widen the gap between vegetation greening and wildlife recovery?

Hieu Do (SID:540914928) Van Duong (SID:530745613)
Cong Thanh Vu (SID:530784058)

2026-07-22

1 Introduction & research question

> **Research question**
>
> **Does post-fire heat exposure widen the gap between vegetation recovery and wildlife recovery?**
> Put simply: does the reassuring "it's green again" signal from satellite NDVI become a weaker proxy for habitat recovery when the years after a fire are unusually hot?

This report tests a practical monitoring problem. After a large fire, satellite greenness can return quickly, but that does not necessarily mean habitat function or animal presence has recovered. The analysis therefore compares three signals for the same places and fire years: fire disturbance, vegetation recovery, and wildlife sightings, then asks whether unusually hot post-fire conditions make those signals diverge.

The analysis unit is the **individual major fire event**: one administrative division in one fire year with at least one recorded fire `>= 10,000 ha`. This event-level design is the report's main analytical innovation. It avoids treating a whole LGA as simply "burnt" or "not burnt" forever, and instead attaches each fire event to its own post-fire heat exposure, vegetation response, and sightings response. Every cached dataset below is built **once**; every later section reuses that same event table so the workflow is reproducible and internally consistent.

In wider context, this is a stress test of the common "Australia is doing well if the landscape looks green again" narrative. A positive result would suggest that satellite recovery alone can give false reassurance under hotter post-fire conditions. A weak or mixed result would still be

useful, because it identifies where the current wildlife data are not strong enough to support national claims.

Executive summary

- The report builds one event-level dataset linking major fires, heat exposure, NDVI recovery, and wildlife sightings.
- The clearest result is that hotter-than-normal early post-fire periods are associated with weaker vegetation recovery.
- The wildlife and recovery-gap results point toward a possible monitoring blind spot, but they remain less certain because sightings data are presence-only, unevenly recorded, and geographically narrow.
- The final claim is cautious: Australia should not be judged as "doing well" from satellite greenness alone.

The report is organised around four questions:

Step	Reader question	Evidence used
1	Where and when did major fires occur?	Fire-history polygons aggregated to division-year events
2	Did vegetation greenness recover after those fires?	NDVI recovery ratio
3	Did wildlife sightings recover in the same places and years?	Sightings-share recovery ratio
4	Does post-fire heat explain a mismatch?	Heat anomaly, recovery gap, statistical tests

The key contribution is not a single map or chart. It is the integration of fire history, gridded climate exposure, satellite vegetation recovery, and species occurrence records into one event-level recovery dataset.

Dataset	Role	Source
Fire history shapefiles, 7 states/territories	disturbance events: location, date, area	NSW (NPWS), QLD (QPWS), VIC (DEECA), SA, TAS, WA (DBCA), NT
`FAO/GAUL/2015/level2` admin-2 boundaries	LGA-level division geometries	Google Earth Engine

Dataset	Role	Source
`AusENDVI-clim_MCD43A4...nc`	vegetation recovery signal (NDVI), 1982-2022	Gap-filled MODIS NDVI climatology
ERA5-Land daily max temperature	post-fire heat exposure, relative to each division's own baseline	Google Earth Engine (`ECMWF/ERA5_LAND/DAILY_AGGR`), 1982-2022
Koala / greater glider / yellow-bellied glider / yellow-tailed black-cockatoo / powerful owl / little lorikeet occurrences	wildlife presence (6 canopy/hollow-dependent indicator species)	Atlas of Living Australia (`galah`), 1990-2026, all 7 states/territories

2 Setup

```
import os
import json
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
import plotly.express as px

DATA_DIR = os.path.join(os.getcwd(), "..", "data")  # report lives in reports/, data lives in
pd.set_option("display.width", 140)

def sig(p, thresh=0.05):
    """So every significance claim below is read live off the current
    p-value rather than typed as a fixed claim in prose."""
    return f"significant (p = {p:.3g} < {thresh})" if p < thresh else f"not significant (p =

def load_admin2_properties(path):
    with open(path, "r") as f:
        gj = json.load(f)
    rows = []
    for i, feature in enumerate(gj["features"]):
        props = feature["properties"]
        rows.append({
            "feature_index": i,
            "ADM1_NAME": props["ADM1_NAME"],
```

```
            "ADM2_NAME": props["ADM2_NAME"],
            "division_id": f"{props['ADM1_NAME']}||{props['ADM2_NAME']}",
        })
    return pd.DataFrame(rows), gj

def filter_geojson(base_geojson, division_ids):
    division_ids = set(division_ids)
    features = []
    for feature in base_geojson["features"]:
        props = feature["properties"]
        division_id = f"{props['ADM1_NAME']}||{props['ADM2_NAME']}"
        if division_id in division_ids:
            feature = dict(feature)
            feature["properties"] = dict(props)
            feature["properties"]["division_id"] = division_id
            features.append(feature)
    return {"type": "FeatureCollection", "features": features}

def vif_table(df, predictors):
    vals = {}
    for target in predictors:
        others = [p for p in predictors if p != target]
        if not others:
            vals[target] = 1.0
            continue
        y = df[target].to_numpy(float)
        X = np.column_stack([np.ones(len(df))] + [df[o].to_numpy(float) for o in others])
        beta = np.linalg.pinv(X.T @ X) @ X.T @ y
        resid = y - X @ beta
        sst = np.sum((y - y.mean()) ** 2)
        r2 = 1 - np.sum(resid ** 2) / sst if sst > 0 else 0
        vals[target] = np.inf if r2 >= 1 else 1 / (1 - r2)
    return vals

class OLSResult:
    def __init__(self, y_name, x_names, n, clusters, params, se, tvals, pvalues, r2, fitted,
        self.y_name = y_name
        self.x_names = x_names
        self.n = n
        self.clusters = clusters
        self.params = params
        self.bse = se
```

```
        self.tvalues = tvals
        self.pvalues = pvalues
        self.rsquared = r2
        self.fitted = fitted
        self.resid = resid

    def summary(self):
        rows = []
        for name in ["const"] + self.x_names:
            rows.append(
                f"{name:24s} coef={self.params[name]: .5f}  "
                f"cluster_se={self.bse[name]: .5f}  t={self.tvalues[name]: .2f}  "
                f"p={self.pvalues[name]:.4g}"
            )
        return "\n".join([
            f"OLS with cluster-robust standard errors by division",
            f"Outcome: {self.y_name}; n={self.n}; clusters={self.clusters}; R^2={self.rsquare
            *rows,
        ])

def cluster_robust_ols(df, y_col, x_cols, cluster_col):
    model_df = df[[y_col, cluster_col] + x_cols].dropna().reset_index(drop=True)
    y = model_df[y_col].to_numpy(float)
    X = np.column_stack([np.ones(len(model_df))] + [model_df[c].to_numpy(float) for c in x_co
    names = ["const"] + x_cols
    n, k = X.shape
    xtx_inv = np.linalg.pinv(X.T @ X)
    beta = xtx_inv @ X.T @ y
    resid = y - X @ beta

    meat = np.zeros((k, k))
    for _, idx in model_df.groupby(cluster_col).groups.items():
        Xg = X[list(idx), :]
        ug = resid[list(idx)][:, None]
        meat += Xg.T @ ug @ ug.T @ Xg
    g = model_df[cluster_col].nunique()
    correction = (g / (g - 1)) * ((n - 1) / (n - k)) if g > 1 and n > k else 1
    cov = correction * xtx_inv @ meat @ xtx_inv
    se = np.sqrt(np.clip(np.diag(cov), 0, np.inf))
    tvals = np.divide(beta, se, out=np.full_like(beta, np.nan), where=se > 0)
    dfree = max(g - 1, 1)
    pvalues = 2 * stats.t.sf(np.abs(tvals), df=dfree)
```

```
    sst = np.sum((y - y.mean()) ** 2)
    r2 = 1 - np.sum(resid ** 2) / sst if sst > 0 else np.nan
    return OLSResult(
        y_col, x_cols, n, g,
        dict(zip(names, beta)), dict(zip(names, se)),
        dict(zip(names, tvals)), dict(zip(names, pvalues)), r2, X @ beta, resid,
    )

def effect_line(label, effect):
    pct = (effect - 1) * 100
    direction = "higher" if pct >= 0 else "lower"
    return f"{label:<26s} x {effect:.3f}   ({abs(pct):.1f}% {direction} relative recovery)"

def residual_diagnostic_plot(models, labels):
    fig, axes = plt.subplots(len(models), 2, figsize=(10, 3.2 * len(models)))
    axes = np.atleast_2d(axes)
    for row, (model, label) in enumerate(zip(models, labels)):
        axes[row, 0].scatter(model.fitted, model.resid, alpha=0.45, s=18)
        axes[row, 0].axhline(0, color="grey", lw=0.8, ls="--")
        axes[row, 0].set_title(f"{label}: residuals vs fitted")
        axes[row, 0].set_xlabel("Fitted value")
        axes[row, 0].set_ylabel("Residual")

        stats.probplot(model.resid, dist="norm", plot=axes[row, 1])
        axes[row, 1].set_title(f"{label}: normal Q-Q")
    plt.tight_layout()
    plt.show()
```

3 Data sources & processing pipeline

Each subsection documents how one cached file in `data/` was built. The raw-data steps (reading multi-gigabyte shapefiles/netCDF, calling Earth Engine, calling the ALA API) are shown as code but marked `eval: false` – they need local raw data and/or authenticated cloud access that this report doesn't assume is present at render time – immediately followed by a **live** cell that loads the resulting cached CSV and reports the real, current numbers.

3.1 Fire history: loading and standardising 7 jurisdictions

Each state/territory publishes its fire-history layer with different column names and CRSes (NSW/QLD: GDA94 lat/lon, VIC: VicGrid94 projected metres, etc.) – reproject everything

to EPSG:4326 and pull out a common `(state, start_date, area_ha, geometry)` schema so they can be combined. The Northern Territory layer carries its own per-record `state` attribute (border-adjacent fires are sometimes attributed to neighbouring QLD/WA/SA), so that attribute is used directly rather than hardcoding "Northern Territory" for every row.

```
FIRE_SHAPEFILES = {
    "New South Wales": os.path.join(DATA_DIR, "NSW_fire_history", "NPWSFireHistory.shp"),
    "Queensland": os.path.join(DATA_DIR, "QLD_fire_history", "Fire_history___QPWS.shp"),
    "Victoria": os.path.join(DATA_DIR, "VIC_fire_history", "gda94_victoriagrid", "esrishape"
                             "whole_of_dataset", "victoria", "FIRE", "FIRE_HISTORY_LASTBURNT
    "South Australia": os.path.join(DATA_DIR, "SA_fire_history", "FIREMGT_FireHistory_GDA2020
    "Tasmania": os.path.join(DATA_DIR, "TAS_fire_history", "list_fire_history_statewide.gdb")
    "Western Australia": os.path.join(DATA_DIR, "WA_fire_history", "DBCA_Fire_History_DBCA_06
    "Northern Territory": os.path.join(DATA_DIR, "NA_fire_history", "NT_FireHistory_SHP.shp")
}

def _naive_dates(series):
    # Concatenating tz-aware dates (e.g. TAS's File Geodatabase) with tz-naive
    # ones forces the combined column to dtype=object, breaking groupby min/max.
    parsed = pd.to_datetime(series, errors="coerce")
    if getattr(parsed.dt, "tz", None) is not None:
        parsed = parsed.dt.tz_localize(None)
    return parsed

def load_nsw(path):
    gdf = gpd.read_file(path).to_crs(4326)
    return gpd.GeoDataFrame({"state": "New South Wales", "start_date": _naive_dates(gdf["Sta
                             "area_ha": gdf["AreaHa"], "geometry": gdf.geometry}, crs=4326)

def load_qld(path):
    gdf = gpd.read_file(path).to_crs(4326)
    return gpd.GeoDataFrame({"state": "Queensland", "start_date": _naive_dates(gdf["ignitionc
                             "area_ha": gdf["area_ha"], "geometry": gdf.geometry}, crs=4326)

def load_vic(path):
    gdf = gpd.read_file(path).to_crs(4326)
    return gpd.GeoDataFrame({"state": "Victoria", "start_date": _naive_dates(gdf["START_DATE"
                             "area_ha": gdf["AREA_HA"], "geometry": gdf.geometry}, crs=4326)

def load_sa(path):
    gdf = gpd.read_file(path).to_crs(4326)
    return gpd.GeoDataFrame({"state": "South Australia", "start_date": _naive_dates(gdf["FIRE
                             "area_ha": gdf["HECTARES"], "geometry": gdf.geometry}, crs=4326
```

```
def load_tas(path):
    gdf = gpd.read_file(path, layer="list_fire_history_statewide").to_crs(4326)  # File Geoda
    return gpd.GeoDataFrame({"state": "Tasmania", "start_date": _naive_dates(gdf["IGN_DATE"])
                             "area_ha": gdf["FIRE_AR_HA"], "geometry": gdf.geometry}, crs=43

def load_wa(path):
    gdf = gpd.read_file(path).to_crs(4326)
    return gpd.GeoDataFrame({"state": "Western Australia", "start_date": _naive_dates(gdf["fi
                             "area_ha": gdf["fih_hectar"], "geometry": gdf.geometry}, crs=43

def load_nt(path):
    # Border-adjacent records may carry a neighbouring state's own attribution --
    # not deduplicated against the QLD/WA/SA loaders above; flagged in Limitations.
    gdf = gpd.read_file(path).to_crs(4326)
    state_map = {"NT": "Northern Territory", "Qld": "Queensland", "QLD": "Queensland",
                 "WA": "Western Australia", "SA": "South Australia"}
    return gpd.GeoDataFrame({"state": gdf["state"].map(lambda s: state_map.get(s, s)),
                             "start_date": _naive_dates(gdf["ignition_d"]),
                             "area_ha": gdf["area_ha"], "geometry": gdf.geometry}, crs=4326)

fires = pd.concat([
    load_nsw(FIRE_SHAPEFILES["New South Wales"]), load_qld(FIRE_SHAPEFILES["Queensland"]),
    load_vic(FIRE_SHAPEFILES["Victoria"]), load_sa(FIRE_SHAPEFILES["South Australia"]),
    load_tas(FIRE_SHAPEFILES["Tasmania"]), load_wa(FIRE_SHAPEFILES["Western Australia"]),
    load_nt(FIRE_SHAPEFILES["Northern Territory"]),
], ignore_index=True)
fires = gpd.GeoDataFrame(fires, geometry="geometry", crs=4326)
fires = fires[fires.geometry.notna() & fires.geometry.is_valid]
print(f"Total fire records: {len(fires)}")
```

3.2 Admin-2 (LGA-level) division boundaries

A small one-time vector pull from Earth Engine (`FAO/GAUL/2015/level2`), cached locally so it only hits Earth Engine once.

```
import ee
import geemap

ADMIN2_CACHE = os.path.join(DATA_DIR, "au_admin2_boundaries.geojson")
REQUIRED_STATES = ["New South Wales", "Victoria", "Queensland", "South Australia",
                   "Tasmania", "Western Australia", "Northern Territory"]
```

```
ee.Initialize(project="project-868d3a54-1ed8-47ec-9ad")
gaul = ee.FeatureCollection("FAO/GAUL/2015/level2")
au_fc = (gaul.filter(ee.Filter.eq("ADM0_NAME", "Australia"))
             .filter(ee.Filter.inList("ADM1_NAME", REQUIRED_STATES)))
admin2 = geemap.ee_to_gdf(au_fc).set_crs(4326, allow_override=True)
admin2.to_file(ADMIN2_CACHE, driver="GeoJSON")
```

```
admin2, admin2_geojson = load_admin2_properties(os.path.join(DATA_DIR, "au_admin2_boundaries
print(f"{len(admin2)} admin-2 divisions loaded, covering: {sorted(admin2['ADM1_NAME'].unique
```

```
563 admin-2 divisions loaded, covering: ['New South Wales', 'Northern Territory', 'Queenslan
```

3.3 Defining "fire-prone" divisions

Fires are spatially joined to divisions on the fire polygon's **centroid** (fast point-in-polygon; fine here since only "which division is this fire mostly in" is needed, not exact boundary overlap). Fire count alone favours Victoria, which has ~440k small recorded burns (median ~11 ha) spread over few divisions, versus NSW/QLD's far fewer but much larger fires. Requiring **both** a minimum fire count *and* a minimum average area per fire targets divisions with frequent, *significant* fire activity, not just lots of small planned burns.

```
import warnings

fire_points = fires.copy()
with warnings.catch_warnings():
    warnings.simplefilter("ignore", category=UserWarning)
    fire_points["geometry"] = fires.geometry.centroid  # only need "roughly which division",

joined = gpd.sjoin(fire_points, admin2, how="left", predicate="within")

division_summary = (
    joined.dropna(subset=["ADM2_NAME"])
    .groupby(["ADM1_NAME", "ADM2_NAME"])
    .agg(fire_count=("area_ha", "size"), total_area_ha=("area_ha", "sum"),
         earliest_fire=("start_date", "min"), latest_fire=("start_date", "max"))
    .reset_index()
)
division_summary["avg_area_ha_per_fire"] = division_summary["total_area_ha"] / division_summa

MIN_FIRE_COUNT = 178     # median fire_count across all divisions with >=1 fire
```

```
MIN_AVG_AREA_HA = 184    # median avg_area_ha_per_fire across the same set
filtered = division_summary[
    (division_summary["fire_count"] >= MIN_FIRE_COUNT) &
    (division_summary["avg_area_ha_per_fire"] >= MIN_AVG_AREA_HA)
].sort_values("fire_count", ascending=False)
filtered.to_csv(os.path.join(DATA_DIR, "fire_prone_divisions.csv"), index=False)
```

```
fire_prone = pd.read_csv(os.path.join(DATA_DIR, "fire_prone_divisions.csv"))
print(f"{len(fire_prone)} fire-prone divisions, by state:")
print(fire_prone.groupby("ADM1_NAME").size().sort_values(ascending=False))
```

```
133 fire-prone divisions, by state:
ADM1_NAME
Western Australia     49
New South Wales       38
Queensland            29
Tasmania               8
South Australia        5
Northern Territory     4
dtype: int64
```

Requiring both a high fire count *and* large average fire size currently skews the list toward NSW/QLD/WA – Victoria's fires are individually small (median ~11 ha), so it drops out entirely at these thresholds; lowering `MIN_AVG_AREA_HA` would bring it back in.

3.4 Major fire events

The raw layers mix routine hazard-reduction burns with genuine wildfires, and at LGA scale almost every fire-prone division has at least one fire recorded in almost every year – so a naive fire count gives virtually no fire-free baseline to compare against. Fires `>= 10,000 ha` are treated as a "major" (landscape-scale) fire, separating real wildfires from routine hazard-reduction blocks.

```
MAJOR_FIRE_HA = 10000  # landscape-scale burn threshold

fire_events = (
    joined.dropna(subset=["ADM2_NAME", "start_date"])
    .merge(filtered[["ADM1_NAME", "ADM2_NAME"]], on=["ADM1_NAME", "ADM2_NAME"], how="inner")
    .assign(year=lambda d: d["start_date"].dt.year, is_major=lambda d: d["area_ha"] >= MAJOR_
    .groupby(["ADM1_NAME", "ADM2_NAME", "year"])
```

```
    .agg(fire_events=("area_ha", "size"), total_area_ha=("area_ha", "sum"),
         max_area_ha=("area_ha", "max"), major_fires=("is_major", "sum"))
    .reset_index()
)
fire_events["year"] = fire_events["year"].astype(int)
fire_events.to_csv(os.path.join(DATA_DIR, "fire_events_by_division_year.csv"), index=False)
```

```
fire_events = pd.read_csv(os.path.join(DATA_DIR, "fire_events_by_division_year.csv"))
major = fire_events[fire_events.major_fires > 0].copy()
print(f"{len(fire_events)} division-year rows; {len(major)} contain at least one fire >= 10,0
      f"across states: {sorted(major['ADM1_NAME'].unique())}")
```

```
6330 division-year rows; 1143 contain at least one fire >= 10,000 ha, across states: ['New So
```

3.5 Vegetation recovery: annual NDVI per division

Each division polygon is rasterised against the NDVI grid with `regionmask` (one boolean mask per division), then averaged over each division's pixels for every year, 1982-2022 – giving one NDVI time series per division rather than per pixel.

```
import xarray as xr
import regionmask

NDVI_NC = os.path.join(DATA_DIR, "AusENDVI-clim_MCD43A4_gapfilled_1982_2022_0.1.0.nc")
NDVI_VAR = "AusENDVI_clim_MCD43A4"

admin2_fire_prone = admin2.merge(filtered[["ADM1_NAME", "ADM2_NAME"]], on=["ADM1_NAME", "ADM2

ds = xr.open_dataset(NDVI_NC, engine="netcdf4")
annual = ds[NDVI_VAR].groupby("time.year").mean(dim="time").load()
mask3D = regionmask.mask_3D_geopandas(admin2_fire_prone, annual.longitude, annual.latitude)
div_annual = annual.where(mask3D).mean(dim=["latitude", "longitude"], skipna=True).compute()

region_states = admin2_fire_prone.loc[mask3D.region.values, "ADM1_NAME"].values
region_names = admin2_fire_prone.loc[mask3D.region.values, "ADM2_NAME"].values
ndvi_by_division = div_annual.to_pandas()
ndvi_by_division.columns = pd.MultiIndex.from_arrays([region_states, region_names], names=["A
ndvi_by_division = ndvi_by_division.T
ndvi_by_division.to_csv(os.path.join(DATA_DIR, "ndvi_annual_by_division.csv"))
```

```
ndvi = pd.read_csv(os.path.join(DATA_DIR, "ndvi_annual_by_division.csv")).set_index(["ADM1_NA
ndvi.columns = ndvi.columns.astype(int)
print(f"NDVI table: {ndvi.shape[0]} divisions x {ndvi.shape[1]} years ({ndvi.columns.min()}-
```

```
NDVI table: 133 divisions x 41 years (1982-2022)
```

3.6 Post-fire heat exposure

The heat data are downloaded from **Google Earth Engine**, using the ERA5-Land daily aggregate product (`ECMWF/ERA5_LAND/DAILY_AGGR`) and the `temperature_2m_max` band. For each year from 1982 to 2022, the pipeline counts days where daily maximum temperature reaches at least 35C (`hot_days_ge35`) and 40C (`hot_days_ge40`), then averages those counts over each fire-prone division. This gives a gridded, division-level heat-exposure measure that is spatially consistent with the LGA-level fire and recovery analysis.

The Earth Engine extraction is shown below for transparency but is not run during report rendering. The rendered report instead reads the cached file `heat_stress_by_division_year.csv`. Division geometries are simplified before upload to reduce the Earth Engine request payload; because ERA5-Land is a coarse climate grid, this simplification is not expected to materially change a division-level annual average.

```
import ee
import geemap

ee.Initialize(project="project-868d3a54-1ed8-47ec-9ad")
admin2_fp = admin2.merge(fire_prone[["ADM1_NAME", "ADM2_NAME"]], on=["ADM1_NAME", "ADM2_NAME"
admin2_fp["geometry"] = admin2_fp.geometry.simplify(0.01, preserve_topology=True)
fc_all = geemap.geopandas_to_ee(admin2_fp)

era5 = ee.ImageCollection("ECMWF/ERA5_LAND/DAILY_AGGR").select("temperature_2m_max")
HOT35_K, HOT40_K = 308.15, 313.15  # 35C / 40C in Kelvin

def per_year(y):
    y = ee.Number(y)
    start = ee.Date.fromYMD(y, 1, 1)
    coll = era5.filterDate(start, start.advance(1, "year"))
    hot35 = coll.map(lambda img: img.gt(HOT35_K)).sum().rename("hot_days_ge35")
    hot40 = coll.map(lambda img: img.gt(HOT40_K)).sum().rename("hot_days_ge40")
    mean_tmax = coll.mean().subtract(273.15).rename("mean_tmax_c")
    combo = hot35.addBands(hot40).addBands(mean_tmax)
    return combo.reduceRegions(collection=fc_all, reducer=ee.Reducer.mean(), scale=11132).map
```

```
all_rows = []
for start_yr in range(1982, 2023, 5):  # chunked into 5-year requests to stay under payload/t
    yrs = list(range(start_yr, min(start_yr + 5, 2023)))
    yearly_fc = ee.FeatureCollection(ee.List(yrs).map(per_year)).flatten()
    all_rows.extend([f["properties"] for f in yearly_fc.getInfo()["features"]])

heat = pd.DataFrame(all_rows)
heat["year"] = heat["year"].astype(int)
heat = heat[["ADM1_NAME", "ADM2_NAME", "year", "hot_days_ge35", "hot_days_ge40", "mean_tmax_
heat.to_csv(os.path.join(DATA_DIR, "heat_stress_by_division_year.csv"), index=False)
```

```
heat = pd.read_csv(os.path.join(DATA_DIR, "heat_stress_by_division_year.csv"))
heat_piv = heat.pivot_table(index=["ADM1_NAME", "ADM2_NAME"], columns="year", values="hot_day
HEAT_BASELINE_YEARS = list(heat_piv.columns)
heat_baseline_mean = heat_piv.mean(axis=1)
print(f"{heat.shape[0]} division-year rows, {heat['ADM2_NAME'].nunique()} divisions, "
      f"{heat['year'].min()}-{heat['year'].max()}, states: {sorted(heat['ADM1_NAME'].unique()
print(f"Heat anomaly baseline: {min(HEAT_BASELINE_YEARS)}-{max(HEAT_BASELINE_YEARS)} availabl
```

```
5453 division-year rows, 133 divisions, 1982-2022, states: ['New South Wales', 'Northern Ter
Heat anomaly baseline: 1982-2022 available-data local baseline
```

3.7 Indicator-species sightings

Six canopy/hollow-dependent species – koala, greater glider, yellow-bellied glider, yellow-tailed black-cockatoo, powerful owl, and little lorikeet – are pulled from the Atlas of Living Australia via `galah`, across all 7 states/territories, 1990-2026. Citizen-science reporting effort has grown enormously nation-wide since the 1990s (more observers, apps like iNaturalist), so each division's sightings are later expressed as a **share of that year's total sightings across all fire-prone divisions** – a division whose wildlife is genuinely declining relative to everywhere else should lose share over time even while the nationwide raw count keeps climbing.

```
import galah

galah.galah_config(email="hieudonguyenthanh2906@gmail.com")  # must be registered at ala.org
states_to_pull = ["New South Wales", "Victoria", "Queensland", "South Australia",
                  "Tasmania", "Western Australia", "Northern Territory"]
indicator_species = [
    "Phascolarctos cinereus",  # Koala
    "Petauroides volans",      # Greater Glider
```

```
    "Petaurus australis",      # Yellow-bellied Glider
    "Zanda funerea",           # Yellow-tailed Black-Cockatoo
    "Ninox strenua",           # Powerful Owl
    "Parvipsitta pusilla",     # Little Lorikeet
]
all_sightings = [
    galah.atlas_occurrences(taxa=indicator_species,
                            filters=["year>=1990", "year<=2026", f"stateProvince={state}"])
    for state in states_to_pull
]
pd.concat(all_sightings, ignore_index=True).to_csv(
    os.path.join(DATA_DIR, "canopy_indicators_all_states.csv"), index=False
)
```

```
animals = pd.read_csv(os.path.join(DATA_DIR, "canopy_indicators_all_states.csv"),
                      usecols=["decimalLatitude", "decimalLongitude", "eventDate", "scienti
animals = animals.dropna(subset=["decimalLatitude", "decimalLongitude", "eventDate"])
animals["eventDate"] = pd.to_datetime(animals["eventDate"], errors="coerce", utc=True)
animals = animals.dropna(subset=["eventDate"])
animals["year"] = animals["eventDate"].dt.year

animals_gdf = gpd.GeoDataFrame(animals, geometry=gpd.points_from_xy(animals["decimalLongitud
animals_joined = gpd.sjoin(animals_gdf, admin2, how="left", predicate="within")
animals_joined = animals_joined.merge(fire_prone[["ADM1_NAME", "ADM2_NAME"]], on=["ADM1_NAME

sightings_by_year = animals_joined.groupby(["ADM1_NAME", "ADM2_NAME", "year"]).size().rename
sightings_by_year.to_csv(os.path.join(DATA_DIR, "animal_sightings_by_division_year.csv"), in
```

```
sightings_raw = pd.read_csv(os.path.join(DATA_DIR, "animal_sightings_by_division_year.csv"))
print(f"{sightings_raw['sightings'].sum()} sightings across {sightings_raw['ADM2_NAME'].nuni
      f"states: {sorted(sightings_raw['ADM1_NAME'].unique())}")
```

```
273844 sightings across 80 divisions, states: ['New South Wales', 'Northern Territory', 'Que
```

4 Building the event-level recovery dataset

This section turns the cached annual datasets into one row per major fire event. Each row contains the fire year, fire severity, vegetation recovery, sightings recovery, and post-fire heat anomaly for the same division.

Feature	Construction	Interpretation
Pre-fire baseline	Mean of `Y-1` and `Y-2`	Local condition before the fire
Fire-year value	Annual value in fire year `Y`	Proxy for the disturbance trough
Recovery value	Best available value from `Y+3`, `Y+4`, `Y+5`	Medium-term post-fire recovery
Recovery ratio	`(recovery - fire-year) / (pre-fire baseline - fire-year)`	Around 1 = back to pre-fire baseline
Heat anomaly	Mean hot days from `Y` to `Y+3` minus the division's own mean annual hot-day count	Early post-fire heat stress relative to local conditions
Recovery gap	`log(NDVI recovery ratio) - log(sightings recovery ratio)`	Positive = vegetation recovers more strongly than wildlife sightings

Method note

The recovery ratio is adapted from Frazier et al. (2018), but it is not a direct copy of their pixel-level LandTrendr method. Here, the disturbance trough is approximated with the annual division-wide fire-year value, and the recovery window uses the best available value from `Y+3` to `Y+5`. The same structure is applied to NDVI and sightings share so the two recovery signals can be compared on a common scale.

Sightings are converted to each division's **share** of that year's total sightings across all fire-prone divisions before the recovery ratio is calculated. This does not remove all observer bias, but it reduces the effect of the nationwide growth in citizen-science records over time.

```
div_year = sightings_raw.groupby(["ADM1_NAME", "ADM2_NAME", "year"])["sightings"].sum().reset
total_per_year = div_year.groupby("year")["sightings"].sum().rename("total_sightings_all_divs
div_year = div_year.merge(total_per_year, on="year")
div_year["share"] = div_year["sightings"] / div_year["total_sightings_all_divs"]
share_piv = div_year.pivot_table(index=["ADM1_NAME", "ADM2_NAME"], columns="year", values="sh
total_sightings_per_div = sightings_raw.groupby(["ADM1_NAME", "ADM2_NAME"])["sightings"].sum

PRE_YEARS, HEAT_POST_LAG, RECOVERY_POST_LAGS, SMOOTH = 2, 3, [3, 4, 5], 1e-6

records = []
for _, r in major.iterrows():
    state, div, year = r["ADM1_NAME"], r["ADM2_NAME"], int(r["year"])
    key = (state, div)
```

```
    if key not in ndvi.index or key not in heat_piv.index:
        continue

    ndvi_row, heat_row = ndvi.loc[key], heat_piv.loc[key]
    pre_years = [year - k for k in range(1, PRE_YEARS + 1)]
    heat_post_years = [year + k for k in range(0, HEAT_POST_LAG + 1)]
    recovery_post_years = [year + k for k in RECOVERY_POST_LAGS]

    pre_ndvi = [v for v in (ndvi_row.get(y) for y in pre_years) if pd.notna(v)]
    y0_ndvi = ndvi_row.get(year)
    post_ndvi = [v for v in (ndvi_row.get(y) for y in recovery_post_years) if pd.notna(v)]
    if not pre_ndvi or pd.isna(y0_ndvi) or not post_ndvi:
        continue
    ndvi_baseline = np.mean(pre_ndvi)
    ndvi_disturbance = ndvi_baseline - y0_ndvi
    if ndvi_disturbance <= 0:
        continue
    ndvi_ratio = (max(post_ndvi) - y0_ndvi) / ndvi_disturbance

    sight_ratio = np.nan
    if key in share_piv.index:
        srow = share_piv.loc[key]
        pre_share = [v for v in (srow.get(y) for y in pre_years) if pd.notna(v)]
        y0_share = srow.get(year)
        post_share = [v for v in (srow.get(y) for y in recovery_post_years) if pd.notna(v)]
        if pre_share and pd.notna(y0_share) and post_share:
            share_baseline = np.mean(pre_share) + SMOOTH
            y0_share_s = y0_share + SMOOTH
            share_disturbance = share_baseline - y0_share_s
            if share_disturbance > 0:
                sight_ratio = (max(post_share) + SMOOTH - y0_share_s) / share_disturbance

    post_heat = [v for v in (heat_row.get(y) for y in heat_post_years) if pd.notna(v)]
    if not post_heat:
        continue
    heat_post_mean = np.mean(post_heat)
    heat_anomaly = heat_post_mean - heat_baseline_mean.get(key, np.nan)

    records.append({
        "ADM1_NAME": state, "ADM2_NAME": div, "fire_year": year,
        "max_area_ha": r["max_area_ha"], "major_fires": r["major_fires"],
        "ndvi_recovery_ratio": ndvi_ratio, "sightings_recovery_ratio": sight_ratio,
```

```
        "heat_anomaly_postfire": heat_anomaly,
        "division_total_sightings": total_sightings_per_div.get(key, 0),
    })

events_all = pd.DataFrame(records)
# RRI can be zero/negative (best post-fire value never exceeds the fire-year value); can't lo
events_all = events_all[events_all["ndvi_recovery_ratio"] > 0].copy()
events_all["log_ndvi_ratio"] = np.log(events_all["ndvi_recovery_ratio"])
events_all["log_severity"] = np.log(events_all["max_area_ha"])

HABITAT_MIN_SIGHTINGS = 200
events = events_all[(events_all["division_total_sightings"] >= HABITAT_MIN_SIGHTINGS)
                    & events_all["sightings_recovery_ratio"].notna()
                    & (events_all["sightings_recovery_ratio"] > 0)].copy()
events["log_sight_ratio"] = np.log(events["sightings_recovery_ratio"])
events["recovery_gap"] = events["log_ndvi_ratio"] - events["log_sight_ratio"]

print(f"{len(events_all)} major-fire events with a usable NDVI recovery ratio + heat exposure
      f"({events_all['ADM2_NAME'].nunique()} divisions, states: {sorted(events_all['ADM1_NAM
print(f"{len(events)} of those also have a valid sightings recovery ratio in a habitat divisi
      f"({events['ADM2_NAME'].nunique()} divisions, states: {sorted(events['ADM1_NAME'].uniq
```

```
400 major-fire events with a usable NDVI recovery ratio + heat exposure (106 divisions, state
53 of those also have a valid sightings recovery ratio in a habitat division (29 divisions, s
```

`events_all` (NDVI-only) and `events` (the habitat/sightings-gated subset, used for every sightings-side and recovery-gap test below) are each built **once** here and reused unchanged for the rest of the report.

The states actually retained in `events` above reflect a biogeographic fact, not a bug: these six indicator species are essentially south-eastern-Australia forest species and are rarely recorded in WA/NT/TAS/SA regardless of fire or NDVI coverage there, so every sightings-side and recovery-gap result below necessarily runs on whichever state(s) currently clear the `>= 200`-sightings bar (see Limitations).

5 Descriptive overview

```
events[["heat_anomaly_postfire", "ndvi_recovery_ratio", "sightings_recovery_ratio", "recovery
```

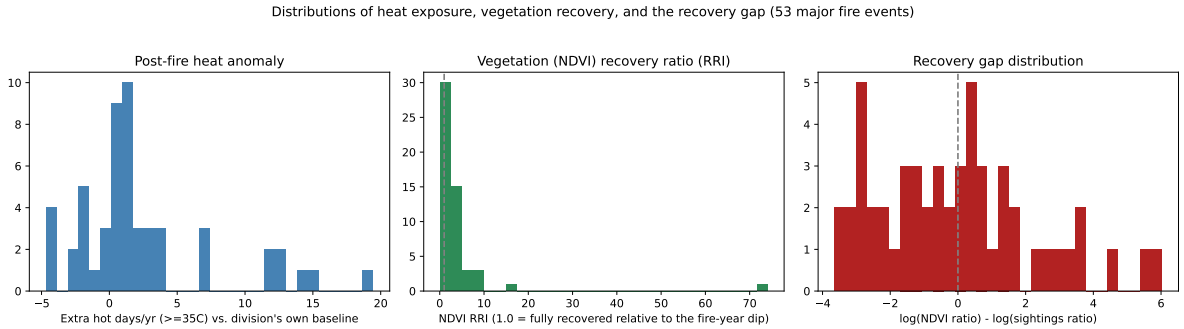
	heat_anomaly_postfire	ndvi_recovery_ratio	sightings_recovery_ratio	recovery_gap
count	53.000000	53.000000	53.000000	53.000000
mean	2.445986	4.306883	11.486999	-0.072387
std	5.239923	10.165986	26.820272	2.345963
min	-4.692032	0.079690	0.013924	-3.646718
25%	-0.037209	1.236241	0.945405	-1.717235
50%	1.309639	2.203394	1.532142	-0.102194
75%	2.842186	3.697023	9.180064	1.461929
max	19.456584	74.078020	162.002857	6.031864

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].hist(events["heat_anomaly_postfire"], bins=30, color="steelblue")
axes[0].set_title("Post-fire heat anomaly")
axes[0].set_xlabel("Extra hot days/yr (>=35C) vs. division's own baseline")

axes[1].hist(events["ndvi_recovery_ratio"], bins=30, color="seagreen")
axes[1].axvline(1.0, color="grey", ls="--")
axes[1].set_title("Vegetation (NDVI) recovery ratio (RRI)")
axes[1].set_xlabel("NDVI RRI (1.0 = fully recovered relative to the fire-year dip)")

axes[2].hist(events["recovery_gap"], bins=30, color="firebrick")
axes[2].axvline(0, color="grey", ls="--")
axes[2].set_title("Recovery gap distribution")
axes[2].set_xlabel("log(NDVI ratio) - log(sightings ratio)")

fig.suptitle(f"Distributions of heat exposure, vegetation recovery, and the recovery gap ({le
plt.tight_layout()
plt.show()
```



6 Interactive map: where does the recovery gap show up?

Each fire-prone division's **mean recovery gap** across all its major-fire events. Red = vegetation recovers substantially better than wildlife (higher false-reassurance risk); blue = the two track each other. The join below matches on **both** `ADM1_NAME` and `ADM2_NAME` (not `ADM2_NAME` alone), since some LGA names recur across state borders (e.g. "Campbelltown (C)" exists in both NSW and SA) and a name-only join could silently misattribute those divisions on the map.

```
div_agg = events.groupby(["ADM1_NAME", "ADM2_NAME"]).agg(
    n_events=("recovery_gap", "size"),
    mean_recovery_gap=("recovery_gap", "mean"),
    mean_heat_anomaly=("heat_anomaly_postfire", "mean"),
    mean_ndvi_ratio=("ndvi_recovery_ratio", "mean"),
    mean_sightings_ratio=("sightings_recovery_ratio", "mean"),
).reset_index()
div_agg["division_id"] = div_agg["ADM1_NAME"] + "||" + div_agg["ADM2_NAME"]

geojson = filter_geojson(admin2_geojson, div_agg["division_id"])

div_agg_labelled = div_agg.rename(columns={
    "ADM1_NAME": "State", "mean_recovery_gap": "Mean recovery gap",
    "mean_heat_anomaly": "Mean post-fire heat anomaly (extra hot days/yr)",
    "mean_ndvi_ratio": "Mean NDVI recovery ratio", "mean_sightings_ratio": "Mean sightings re
    "n_events": "Number of major fire events",
})

fig_map = px.choropleth_map(
    div_agg_labelled, geojson=geojson, locations="division_id", featureidkey="properties.divi
    color="Mean recovery gap", color_continuous_scale="RdBu_r", color_continuous_midpoint=0,
    hover_name="ADM2_NAME",
    hover_data={"State": True, "Mean recovery gap": ":.2f",
                "Mean post-fire heat anomaly (extra hot days/yr)": ":.1f",
                "Mean NDVI recovery ratio": ":.2f", "Mean sightings recovery ratio": ":.2f",
                "Number of major fire events": True},
    map_style="carto-positron", zoom=3.2, center={"lat": -28, "lon": 145}, opacity=0.75, heig
)
fig_map.update_layout(
    title=dict(text="Mean post-fire recovery gap by fire-prone division<br>"
                    "<sup>Red = vegetation recovery exceeds wildlife recovery; Blue = the tw
               x=0.02, y=0.98),
    margin=dict(l=0, r=0, t=70, b=0),
```

```
    coloraxis_colorbar=dict(title="Mean recovery gap<br>(+ = vegetation<br>outpaces wildlife)
)
fig_map
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

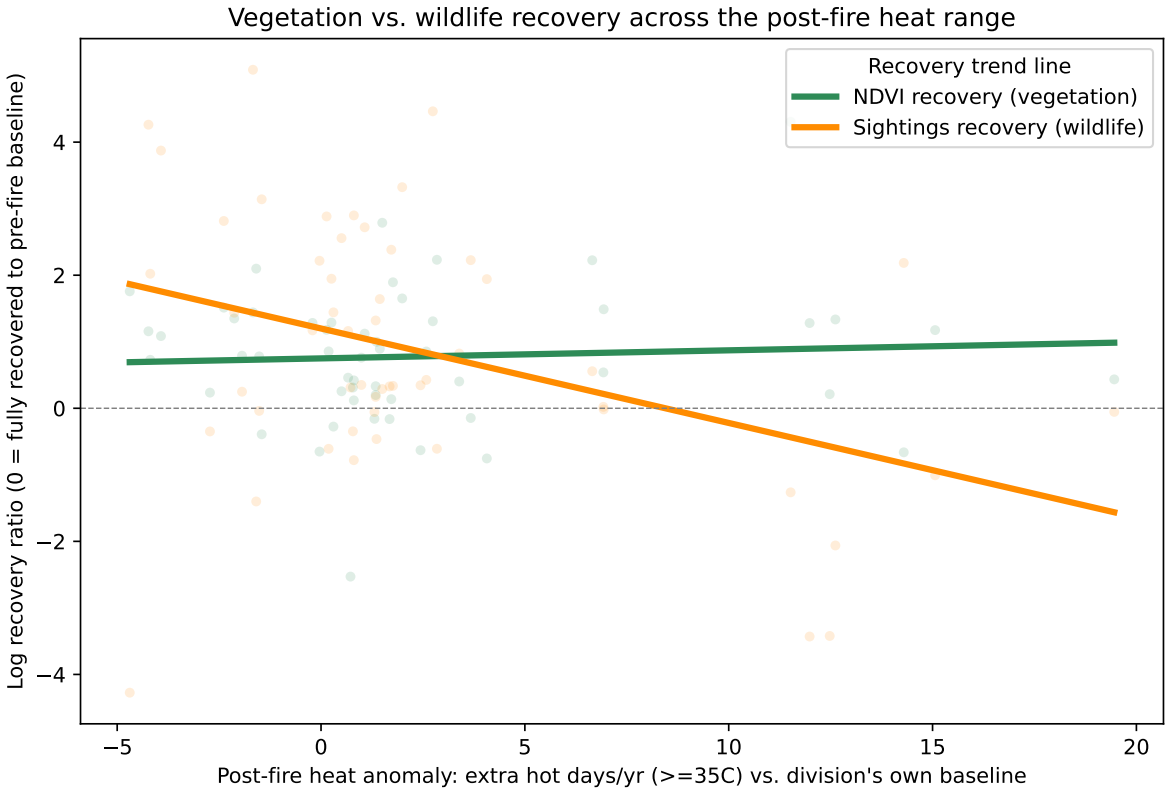
7 Visual evidence

7.1 Diverging recovery lines

Both recovery ratios (log scale, 0 = back to baseline) plotted against post-fire heat exposure, with a trend line fit to each.

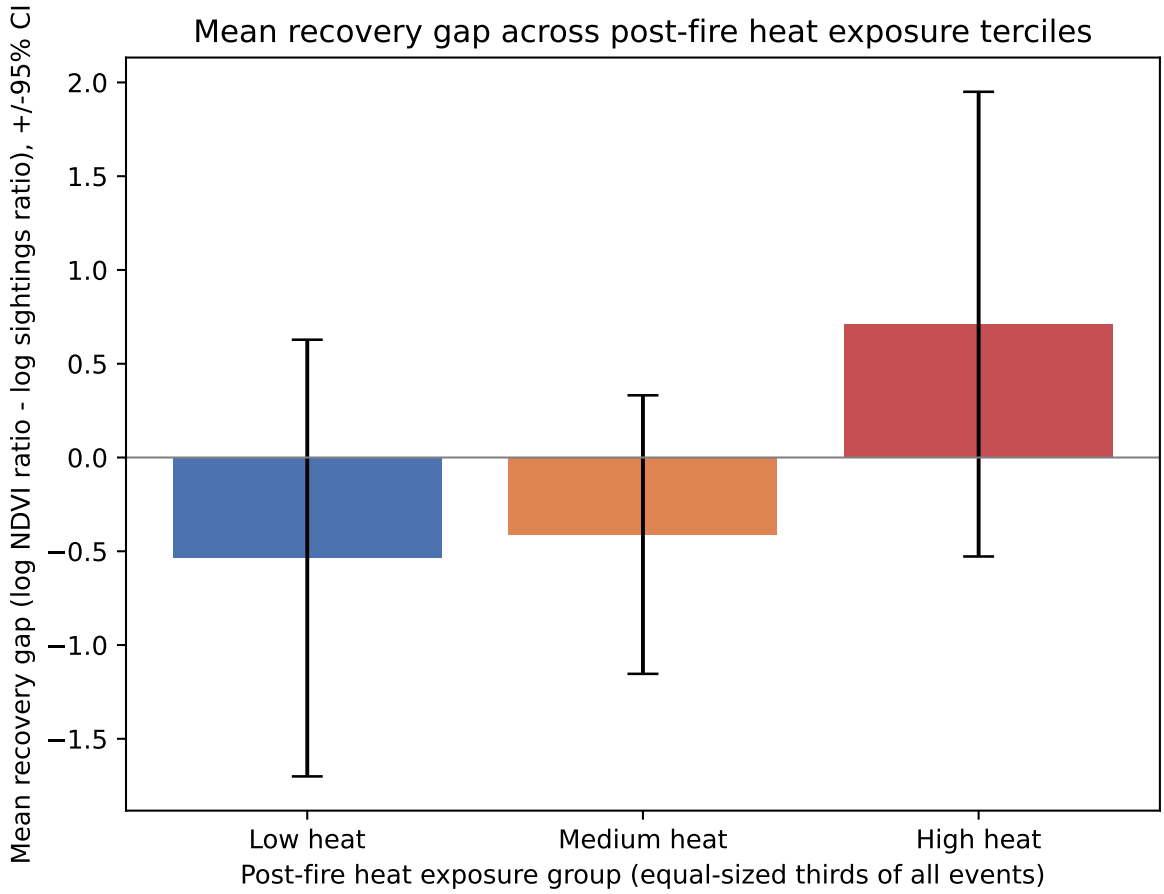
```
fig, ax = plt.subplots(figsize=(8, 5.5))
xs = np.linspace(events["heat_anomaly_postfire"].min(), events["heat_anomaly_postfire"].max()
z_ndvi = np.polyfit(events["heat_anomaly_postfire"], events["log_ndvi_ratio"], 1)
z_sight = np.polyfit(events["heat_anomaly_postfire"], events["log_sight_ratio"], 1)

ax.scatter(events["heat_anomaly_postfire"], events["log_ndvi_ratio"], alpha=0.15, s=14, color
ax.scatter(events["heat_anomaly_postfire"], events["log_sight_ratio"], alpha=0.15, s=14, colo
ax.plot(xs, np.polyval(z_ndvi, xs), color="seagreen", lw=3, label="NDVI recovery (vegetation)
ax.plot(xs, np.polyval(z_sight, xs), color="darkorange", lw=3, label="Sightings recovery (wil
ax.axhline(0, color="grey", lw=0.7, ls="--")
ax.set_xlabel("Post-fire heat anomaly: extra hot days/yr (>=35C) vs. division's own baseline"
ax.set_ylabel("Log recovery ratio (0 = fully recovered to pre-fire baseline)")
ax.set_title("Vegetation vs. wildlife recovery across the post-fire heat range")
ax.legend(title="Recovery trend line", loc="upper right")
plt.tight_layout()
plt.show()
```



7.2 The gap by heat tercile

```
events["heat_tercile"] = pd.qcut(events["heat_anomaly_postfire"], 3, labels=["Low heat", "Mec
tercile = events.groupby("heat_tercile", observed=True).agg(
    mean_gap=("recovery_gap", "mean"), se_gap=("recovery_gap", lambda x: x.std() / np.sqrt(le
)
fig, ax = plt.subplots(figsize=(6.5, 5))
ax.bar(tercile.index, tercile["mean_gap"], yerr=1.96 * tercile["se_gap"], capsize=6,
       color=["#4c72b0", "#dd8452", "#c44e52"])
ax.axhline(0, color="grey", lw=0.8)
ax.set_xlabel("Post-fire heat exposure group (equal-sized thirds of all events)")
ax.set_ylabel("Mean recovery gap (log NDVI ratio - log sightings ratio), +/-95% CI")
ax.set_title("Mean recovery gap across post-fire heat exposure terciles")
plt.tight_layout()
plt.show()
tercile
```



heat_tercile	mean_gap	se_gap
Low heat	-0.536178	0.594045
Medium heat	-0.410984	0.378890
High heat	0.711189	0.632145

8 Statistical testing

The statistical section is designed to separate visual patterns from evidence. It uses a rank-based test, a model-based test with clustered standard errors, and a bootstrap robustness check. The tests are applied first to vegetation and sightings recovery separately, then to the combined **recovery gap**. Every number below is computed live from the current event table rather than typed into the prose.

8.1 Assumption checks before testing

The variables are not ideal for a single simple parametric test. Heat exposure is an anomaly count, fire severity spans orders of magnitude, and recovery ratios can be strongly skewed. For that reason, the report uses:

- **Spearman correlation** for the first pass, because it tests whether higher heat tends to correspond to higher/lower recovery without assuming normally distributed variables or a strictly linear relationship.
- **Log-transformed recovery ratios** for OLS, because the response variables are ratios and proportional changes are easier to interpret on the log scale.
- **Cluster-robust standard errors by division**, because several fire events can occur in the same division and those observations should not be treated as fully independent.
- **Cluster bootstrap by division** for the recovery-gap slope, because the sightings-side sample is smaller and may be sensitive to individual high-recording divisions.

```
assumption_summary = pd.DataFrame({
    "variable": ["heat anomaly", "NDVI recovery ratio", "sightings recovery ratio", "recovery
    "n": [
        events["heat_anomaly_postfire"].notna().sum(),
        events_all["ndvi_recovery_ratio"].notna().sum(),
        events["sightings_recovery_ratio"].notna().sum(),
        events["recovery_gap"].notna().sum(),
        events_all["max_area_ha"].notna().sum(),
    ],
    "skewness": [
        stats.skew(events["heat_anomaly_postfire"], nan_policy="omit"),
        stats.skew(events_all["ndvi_recovery_ratio"], nan_policy="omit"),
        stats.skew(events["sightings_recovery_ratio"], nan_policy="omit"),
        stats.skew(events["recovery_gap"], nan_policy="omit"),
        stats.skew(events_all["max_area_ha"], nan_policy="omit"),
    ],
    "min": [
        events["heat_anomaly_postfire"].min(),
        events_all["ndvi_recovery_ratio"].min(),
        events["sightings_recovery_ratio"].min(),
        events["recovery_gap"].min(),
        events_all["max_area_ha"].min(),
    ],
    "max": [
        events["heat_anomaly_postfire"].max(),
        events_all["ndvi_recovery_ratio"].max(),
        events["sightings_recovery_ratio"].max(),
```

```
        events["recovery_gap"].max(),
        events_all["max_area_ha"].max(),
    ],
})
assumption_summary
```

	variable	n	skewness	min	max
0	heat anomaly	53	1.417830	-4.692032	1.945658e+01
1	NDVI recovery ratio	400	9.953323	0.010336	2.182464e+02
2	sightings recovery ratio	53	4.090474	0.013924	1.620029e+02
3	recovery gap	53	0.623405	-3.646718	6.031864e+00
4	fire severity	400	4.736460	10039.633241	1.889778e+06

These diagnostics justify using Spearman as the headline bivariate test and log-space OLS as a secondary, controlled model. The tests below therefore do not rely on raw recovery ratios being normally distributed.

8.2 Do the individual recovery ratios respond to heat?

Spearman rank correlation (no linearity/distribution assumptions):

```
r1, p1 = stats.spearmanr(events_all["heat_anomaly_postfire"], events_all["ndvi_recovery_ratio
r2, p2 = stats.spearmanr(events["heat_anomaly_postfire"], events["sightings_recovery_ratio"])
print(f"heat anomaly vs NDVI recovery ratio:       rho={r1:.3f}, p={p1:.3g}, n={len(events_al
print(f"heat anomaly vs sightings recovery ratio:  rho={r2:.3f}, p={p2:.3g}, n={len(events)}"
```

```
heat anomaly vs NDVI recovery ratio:       rho=-0.136, p=0.00644, n=400
heat anomaly vs sightings recovery ratio:  rho=-0.295, p=0.0319, n=53
```

OLS regression with cluster-robust standard errors (clustered by division, since events within the same division aren't independent): `log(recovery ratio) ~ heat_anomaly + log(severity)`.

```
X = events_all[["heat_anomaly_postfire", "log_severity"]].copy()
X = (X - X.mean()) / X.std(ddof=0)
vif = vif_table(X, ["heat_anomaly_postfire", "log_severity"])
print("VIF (collinearity check):", vif)
```

```
VIF (collinearity check): {'heat_anomaly_postfire': 1.0223889824668444, 'log_severity': 1.02
```

```
m1 = cluster_robust_ols(events_all, "log_ndvi_ratio",
                        ["heat_anomaly_postfire", "log_severity"], "ADM2_NAME")
print(m1.summary())
```

```
OLS with cluster-robust standard errors by division
Outcome: log_ndvi_ratio; n=400; clusters=106; R^2=0.009
const                    coef= 0.41395  cluster_se= 0.48466  t= 0.85  p=0.395
heat_anomaly_postfire    coef=-0.01402  cluster_se= 0.00598  t=-2.35  p=0.02088
log_severity             coef= 0.01235  cluster_se= 0.04651  t= 0.27  p=0.7912
```

```
m2 = cluster_robust_ols(events, "log_sight_ratio",
                        ["heat_anomaly_postfire", "log_severity"], "ADM2_NAME")
print(m2.summary())
```

```
OLS with cluster-robust standard errors by division
Outcome: log_sight_ratio; n=53; clusters=29; R^2=0.170
const                    coef= 4.29928  cluster_se= 1.96430  t= 2.19  p=0.03712
heat_anomaly_postfire    coef=-0.11086  cluster_se= 0.06987  t=-1.59  p=0.1238
log_severity             coef=-0.30367  cluster_se= 0.20245  t=-1.50  p=0.1448
```

VIF ~1.02 for both predictors – heat exposure and fire severity are not confounded with each other, so their coefficients are independently interpretable. **Post-fire heat anomaly is significant ($p = 0.0209 < 0.05$) for NDVI (n=400) and not significant ($p = 0.124 >= 0.05$) for sightings (n=53)** in the current data.

A division experiencing **20 more hot days/yr than its own long-run average** in the recovery window after a major fire is associated with:

```
ndvi_effect = np.exp(m1.params["heat_anomaly_postfire"] * 20)
sight_effect = np.exp(m2.params["heat_anomaly_postfire"] * 20)
print(effect_line("NDVI recovery ratio", ndvi_effect))
print(effect_line("Sightings recovery ratio", sight_effect))
```

```
NDVI recovery ratio        x 0.755   (24.5% lower relative recovery)
Sightings recovery ratio   x 0.109   (89.1% lower relative recovery)
```

8.3 Does the vegetation-vs-sightings recovery gap widen with heat?

Same two tests, applied directly to the **gap** (`recovery_gap = log_ndvi_ratio - log_sight_ratio`), plus one robustness check on the regression slope.

Spearman rank correlation:

```
r3, p3 = stats.spearmanr(events["heat_anomaly_postfire"], events["recovery_gap"])
print(f"Spearman(heat anomaly, recovery gap): rho={r3:.3f}, p={p3:.4g}, n={len(events)}")
```

```
Spearman(heat anomaly, recovery gap): rho=0.282, p=0.04062, n=53
```

OLS, cluster-robust SEs, on the gap directly:

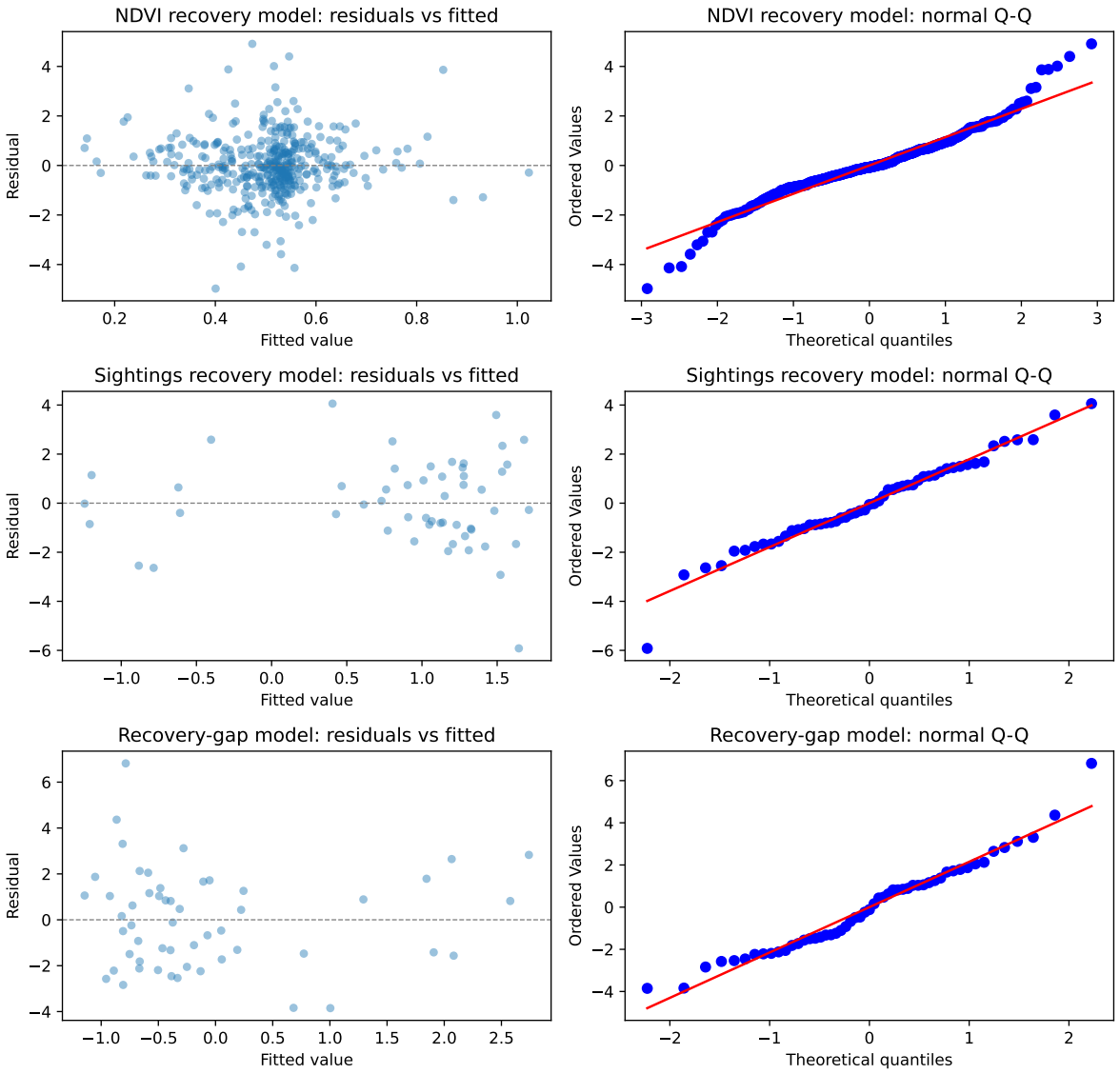
```
m3 = cluster_robust_ols(events, "recovery_gap",
                        ["heat_anomaly_postfire", "log_severity"], "ADM2_NAME")
print(m3.summary())
```

```
OLS with cluster-robust standard errors by division
Outcome: recovery_gap; n=53; clusters=29; R^2=0.170
const                    coef=-5.88093  cluster_se= 2.81127  t=-2.09  p=0.04564
heat_anomaly_postfire    coef= 0.09959  cluster_se= 0.07685  t= 1.30  p=0.2056
log_severity             coef= 0.53204  cluster_se= 0.28044  t= 1.90  p=0.06816
```

8.4 Model diagnostic plots

The residual plots below check whether the OLS results are being driven by obvious non-linearity or extreme residual structure. The Q-Q plots are included as a diagnostic rather than a strict pass/fail test: with cluster-robust standard errors and the bootstrap check below, exact normal residuals are not required for the main interpretation.

```
residual_diagnostic_plot(
    [m1, m2, m3],
    ["NDVI recovery model", "Sightings recovery model", "Recovery-gap model"],
)
```



Cluster bootstrap on the slope (resampling whole divisions, not individual events, 2,000 times – checks the regression result isn't an artefact of one or two divisions with unusually many events):

```
rng = np.random.default_rng(42)
divs = events["ADM2_NAME"].unique()
boot_slopes = []
for _ in range(2000):
    sample_divs = rng.choice(divs, size=len(divs), replace=True)
    boot_df = pd.concat([events[events.ADM2_NAME == d] for d in sample_divs], ignore_index=T
```

```
    boot_slopes.append(np.polyfit(boot_df["heat_anomaly_postfire"], boot_df["recovery_gap"],
boot_slopes = np.array(boot_slopes)
ci_lo, ci_hi = np.percentile(boot_slopes, [2.5, 97.5])
boot_pos_pct = (boot_slopes > 0).mean() * 100
print(f"bootstrap slope: mean={boot_slopes.mean():.4f}, 95% CI=[{ci_lo:.4f}, {ci_hi:.4f}]")
print(f"fraction of 2,000 resamples with a positive slope: {boot_pos_pct:.1f}%")
```

```
bootstrap slope: mean=0.1416, 95% CI=[-0.0548, 0.3531]
fraction of 2,000 resamples with a positive slope: 90.6%
```

In the current data, the gap is significant ($p = 0.0406 < 0.05$) on the Spearman test and not significant ($p = 0.206 >= 0.05$) on the cluster-robust regression, with 91% of bootstrap resamples agreeing on a positive slope. This is best read as suggestive rather than definitive evidence. The rank test and bootstrap direction support the idea that heat may widen the vegetation-vs-wildlife recovery gap, but the cluster-robust model does not clear the conventional 0.05 threshold, so the report avoids making a causal or settled national claim.

Assumption note: Spearman and the cluster bootstrap make essentially no distributional assumptions and are the more robust readings here; the OLS models add linearity-in-log-space (a standard, defensible choice for ratio data) and rely on cluster-robust standard errors, which need more independent divisions than the sightings-side sample currently has (VIF confirms no multicollinearity, so that part of the model is not the concern). Treat the NDVI-heat relationship as the strongest statistical result, and the gap result as an important but less certain stress-test signal.

9 Limitations

- **Correlational, not causal.** Heat, fire severity, drought, and remoteness are entangled; this cannot rule out a third factor (e.g. drought, which drives both worse fire years and low NDVI) acting on both sides at once.
- **Sightings are presence-only and effort-biased.** Reporting effort has grown enormously nation-wide since the 1990s; the share-of-year-total normalisation and the >= 200-sightings habitat filter reduce but do not eliminate this. Treat the sightings result as "recorded presence", not population size.
- **The sightings-side and recovery-gap tests currently run on whichever state(s) clear the >= 200-sightings habitat filter** – printed live above. The six indicator species are essentially south-eastern- Australia forest species and are almost never recorded in WA/NT/TAS/SA regardless of how much fire/NDVI data exists there, so this is a biogeographic fact rather than a processing bug – but it does mean the sightings-side

and recovery-gap conclusions cannot be read as claims about Australian wildlife broadly, only about the state(s) where these particular species occur in numbers.

- **The 10,000 ha "major fire" threshold, and the `MIN_FIRE_COUNT` / `MIN_AVG_AREA_HA` fire-prone-division thresholds, are design choices**, not physical constants. Victoria's ~440k small recorded burns (median ~11 ha) currently fail the average-area threshold and drop out of the fire-prone list entirely; lowering `MIN_AVG_AREA_HA` would bring it back in.
- **The recovery ratio is an adapted RRI, not the original Frazier et al. (2018) formula.** Their method locates the disturbance trough precisely from per-pixel LandTrendr segmentation of Landsat and uses a fixed Y+4/Y+5 recovery window; here the "fire-year" value is the annual division-wide composite for that calendar year (a noisier proxy for the true trough), and the recovery window is the best available of Y+3/Y+4/Y+5 rather than fixed Y+4/Y+5, to keep 2019-onward fires (including Black Summer) inside the NDVI data's 2022 cutoff.
- **The RRI's disturbance-magnitude requirement (a measurable dip in the fire year) sharply cuts the usable sample**, especially for sightings – see the live counts printed above (`events_all` vs `events`). This is the main reason the direct NDVI-heat and sightings-heat relationships can trade places in significance as more data/states are added, and why the gap-focused tests carry meaningful sampling uncertainty.
- **The heat-exposure window (`fire_year` to `fire_year+3`) does not exactly match the recovery-ratio window** (best available of Y+3/Y+4/Y+5). This keeps the original result unchanged and captures early post-fire heat stress, but it should be described as an adapted design choice rather than a published formula copied verbatim from the literature.
- **Multiple major fires can overlap** within a division's post-fire window (a second fire before the recovery year used); this event table does not exclude that case – it measures recovery as observed, whatever happened in between.
- **Fire-to-division spatial join uses fire-polygon centroids**, and the Northern Territory layer's border-adjacent records are not deduplicated against the QLD/WA/SA loaders' own records – a small double-counting risk near state borders that this report flags rather than resolves.

10 Answering the research question

> **! Main judgement**
>
> Australia should not be judged as "doing well" from satellite greenness alone. The analysis shows that hotter early post-fire periods are linked to weaker vegetation recovery, while the wildlife evidence is too limited to confidently confirm that animals recover alongside vegetation.

The answer is mixed, but informative. The data do not support a simple "Australia is doing well" story if recovery is judged only by vegetation greenness. At the same time, the wildlife evidence is not yet strong enough to claim a definitive national recovery gap. The value of the analysis is that it shows where the apparent recovery signal is strong, where it is fragile, and where better monitoring data are needed.

- **Clear finding:** hotter-than-normal early post-fire periods are associated with weaker NDVI recovery after major fires (significant ($p = 0.0209 < 0.05$) in the cluster-robust model, n=400). So "green again" is not guaranteed under hotter post-fire conditions.
- **Wildlife signal:** sightings-share recovery points in the same negative direction in the simple rank test, but the model-based result is not significant ($p = 0.124 >= 0.05$) (n=53). This means the wildlife evidence should be treated as indicative rather than conclusive.
- **Gap signal:** the vegetation-vs-wildlife gap is significant ($p = 0.0406 < 0.05$) on the Spearman test and not significant ($p = 0.206 >= 0.05$) in the cluster-robust model. This suggests a possible blind spot, but the uncertainty is still too large for a definitive causal claim.

The report's contribution is a testable monitoring framework: combine fire history, post-fire heat, NDVI recovery, and species sightings before deciding whether a burnt landscape has genuinely recovered.

11 Appendix A: feature definitions

Term	Definition
Fire-prone division	An admin-2 (LGA) division passing both `fire_count >= MIN_FIRE_COUNT` and `avg_area_ha_per_fire >= MIN_AVG_AREA_HA` – frequent *and* individually significant fire activity, not just lots of small planned burns.
Major fire event	A division-year with at least one recorded fire `>= 10,000 ha` – separates landscape-scale wildfires from routine hazard-reduction burns.
Pre-fire baseline	The mean of the two years immediately before the fire year (Y-1, Y-2), computed separately for NDVI and for sightings share.

Term	Definition
Fire-year value (Y0)	The NDVI/sightings-share value in the fire year itself – stands in for the "disturbance trough" that Frazier et al. (2018) locate precisely from per-pixel LandTrendr segmentation; here it is the annual division-wide composite for that calendar year, a noisier proxy for the same idea.
Post-fire recovery window	The best available value among Y+3, Y+4, Y+5 – adapted from the paper's fixed Y+4/Y+5 window so fires from 2019 onward (including Black Summer) are not dropped by the NDVI data's 2022 cutoff.
NDVI/sightings recovery ratio (RRI)	`RRI = (max(value over the recovery window) - value(Y0)) / (mean(value(Y-1), value(Y-2)) - value(Y0))`. ~1.0 = recovered back to the pre-fire baseline relative to how hard the fire year was hit; below 1 = still short of that; above 1 = overshot it. Only defined when the disturbance magnitude is positive.
Sightings share	A division's sightings in a given year, divided by total sightings across all fire-prone divisions that year – a partial correction for the nationwide growth in citizen-science reporting effort over time.
Heat anomaly (post-fire)	`mean(hot days >=35C from Y to Y+3) - that division's available-data mean annual hot-day count` – measured relative to each division's own baseline, so it captures "unusually hot for this place" rather than just "a hot place."
Recovery gap	`log(NDVI recovery ratio) - log(sightings recovery ratio)`. Zero = vegetation and wildlife recovered by the same proportional amount; positive = vegetation recovered relatively better than wildlife.
Fire severity (`log_severity`)	`log(maximum single fire area in hectares recorded in that division-year)` – log-transformed because fire size spans several orders of magnitude; a regression control variable.

Term	Definition
Habitat division filter	Only divisions with `>= 200` total sightings across the whole record are included in sightings-based tests, to exclude LGAs where these species barely occur and the sightings ratio would just be noise.
Heat tercile	Events split into three equal-sized groups (Low/Medium/High) by post-fire heat anomaly, used for the tercile chart above.

12 References

Frazier, R. J., Coops, N. C., Wulder, M. A., Hermosilla, T., & White, J. C. (2018). Analyzing spatial and temporal variability in short-term rates of post-fire vegetation return from Landsat time series. *Remote Sensing of Environment, 205*, 32-45. https://doi.org/10.1016/j.rse.2017.11.007

Sillmann, J., Kharin, V. V., Zhang, X., Zwiers, F. W., & Bronaugh, D. R. (2013). Climate extremes indices in the CMIP5 multimodel ensemble: Part 1. Model evaluation in the present climate. *Journal of Geophysical Research: Atmospheres, 118*(4), 1716-1733. https://doi.org/10.1002/jgrd.50203

World Meteorological Organization. (2017). *WMO Guidelines on the Calculation of Climate Normals* (WMO-No. 1203). https://library.wmo.int/idurl/4/55797